

## **FINAL PROJECT REPORT: SMART NURSERY GUARDIAN**

### **TEAM 5**

| Students name:                 | Id    |
|--------------------------------|-------|
| 1:Ahmed Abouelwafa Abdelmonsef | 7013E |
| 2:Abdulrahman Mohamed Reza     | 7077E |
| 3:Youssef Mohamed Mounir       | 7105E |
| 4:Ahmed Mohamed Abo elenin     | 7049E |

## 1. Executive Summary & Project Overview

Smart Nursery Guardian is an IoT monitoring setup built to help parents track their baby's safety and status in real time. It automates daily care routines like checking room conditions or detecting crying by linking environmental sensors and audio processing directly to hardware actions (servo rocking, alerts, and video responses).

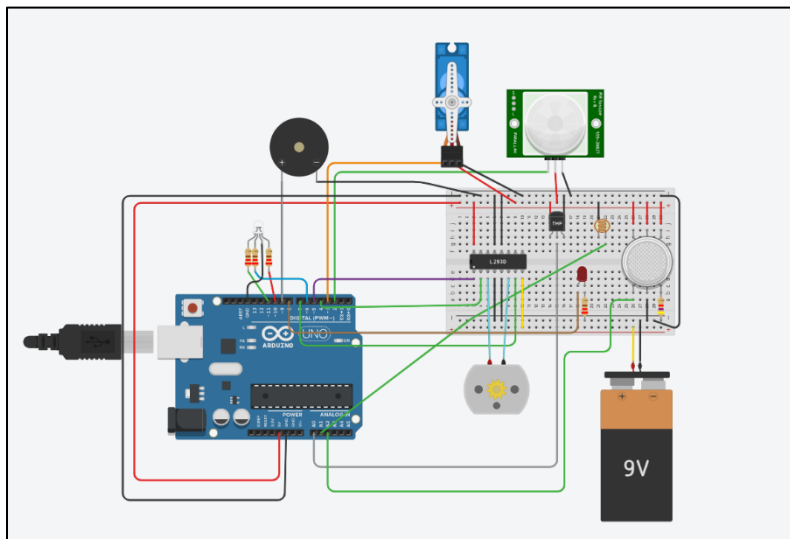
The system split responsibility across two computing tiers:

- **High-Level Intelligence Unit (Python App on Laptop):** Handles real-time audio acquisition, background noise filtering, spectral processing, and Machine Learning classification to categorize baby cries into specific needs (Hungry, Tired, Discomfort).
  - **Low-Level Actuation & Sensing Unit (STM32 Microcontroller):** Continuously samples ambient room conditions (Temperature, Gas/Smoke, Motion, and Light) to control local hardware outputs (Fan, Servo rocking mechanism, Buzzer, and LEDs) in real-time.
  - **Alert System:** Features an automated IoT emergency notification pathway via Telegram to send instant safety alerts for high-priority hazards like gas leaks.
- 

## 2. Hardware Development Methodology (Step-by-Step)

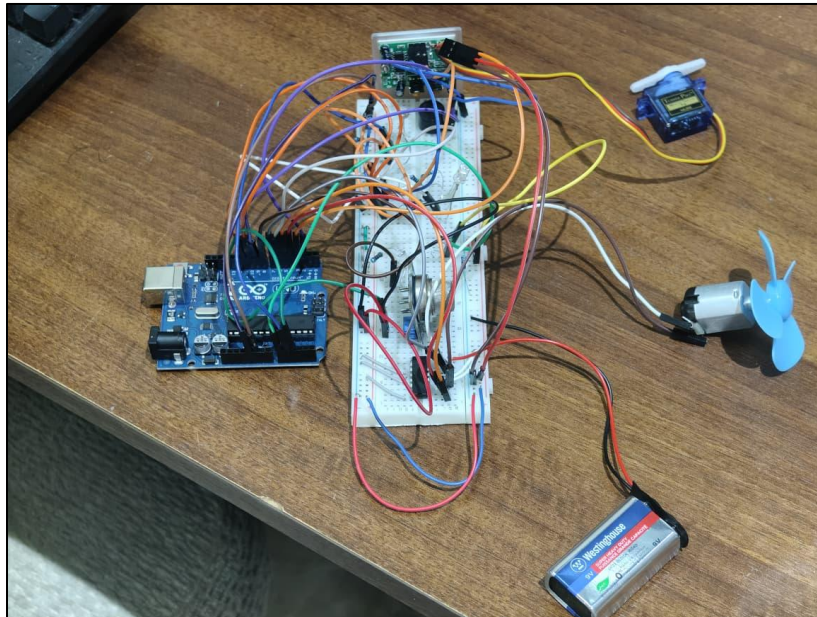
### Step 1: Concept Simulation on Tinkercad (Arduino-based)

- Modeled the core circuit architecture using an Arduino Uno on Tinkercad.
- Simulated multi-sensor data acquisition (PIR motion, LDR light, MQ gas, and Temperature sensors).
- Verified conditional logic for motor drivers, micro-servo crib-rocking actuation, buzzer alert mechanisms, and RGB status lighting prior to physical assembly.



## **Step 2: Physical Hardware Prototyping on Breadboard (Arduino)**

- Constructed the first physical prototype on a breadboard using an Arduino development board.
- Validated electrical connections, pin mapping, power distribution, and current draws for motors and sensors.
- Calibrated analog sensor thresholds (Gas ADC levels and LDR dark/light thresholds) under real-world operating conditions.



---

## **Step 3: STM32 Microcontroller Pin Mapping & Hardware Adaptation**

- Ported the circuit architecture from Arduino to the STM32 (Blue Pill platform).
- Carefully assigned STM32 pins for hardware interfaces, ensuring optimal allocation for:
  - Analog-to-Digital Converter (ADC) channels for Gas and LDR sensors.
  - Universal Asynchronous Receiver-Transmitter (UART) pins (PA9/PA10) for serial communication.
  - Pulse Width Modulation (PWM) pins for Servo motor rocking.
  - Digital GPIOs for PIR motion detection, Buzzer activation, and Fan control via motor drivers.

STM32 Pinout & Connection Mapping Table

| Component / Subsystem       | STM32 Pin  | Signal Type & Function                           |
|-----------------------------|------------|--------------------------------------------------|
| NTC Temperature Sensor      | PA0        | Analog Input (via Voltage Divider)               |
| LDR Light Sensor            | PA1        | Analog Input (via Voltage Divider)               |
| MQ Gas Sensor               | PA2        | Analog Input (via 5V-to-3.3V Voltage Divider)    |
| Micro Servo Motor           | PA6        | PWM Output (Crib Rocking Mechanism)              |
| RGB LED (Red)               | PB0        | Digital Output (System Status Indicator)         |
| RGB LED (Green)             | PB1        | Digital Output (System Status Indicator)         |
| Active Buzzer               | PB5        | Digital Output (Gas Alert & Urgent Tired Cry)    |
| DC Fan Driver (Enable/PWM)  | PB6        | PWM Output (Fan Speed Control)                   |
| RGB LED (Blue)              | PB10       | Digital Output (System Status Indicator)         |
| PIR Motion Sensor           | PB12       | Digital Input (Baby Motion Detection)            |
| DC Fan Driver (Direction 1) | PB13       | Digital Output (L298N/Driver Control)            |
| DC Fan Driver (Direction 2) | PB14       | Digital Output (L298N/Driver Control)            |
| Room Lighting LED           | PB15       | Digital Output (Automated Night Light)           |
| UART Serial (TX / RX)       | PA9 / PA10 | Serial Communication (FTDI / USB to Laptop COM9) |

### Connection Notes

#### 1. Gas Sensor Voltage Level Shifting Protection (5V to 3.3V):

- The MQ Gas sensor operates on 5V and outputs an analog signal up to 5V ,Since the STM32 ADC pins operate on a maximum safe voltage of 3.3V, a **Voltage Divider circuit** was added at the output of the gas sensor.
- **Circuit Setup:** A 10k ohm resistor is connected in series from the MQ output pin, and a 20k ohm resistor is connected to Ground. The analog junction point between the two resistors is connected directly to pin **PA2**, effectively scaling down the maximum 5V signal to a safe 3.33V limit for the STM32 ADC.

#### 2. Analog Signal Conditioning (NTC & LDR):

- Both the **NTC thermistor** (PA0) and the **LDR light sensor** (PA1) are wired in individual **Voltage Divider configurations** with fixed pull-down resistors to convert continuous resistance variations into measurable voltage levels for the STM32 12-bit ADC.

## Step 4: Altium Designer Custom Library & PCB Architecture

- Created custom schematic symbols (.SchLib) and PCB footprints (.PcbLib) for all used components

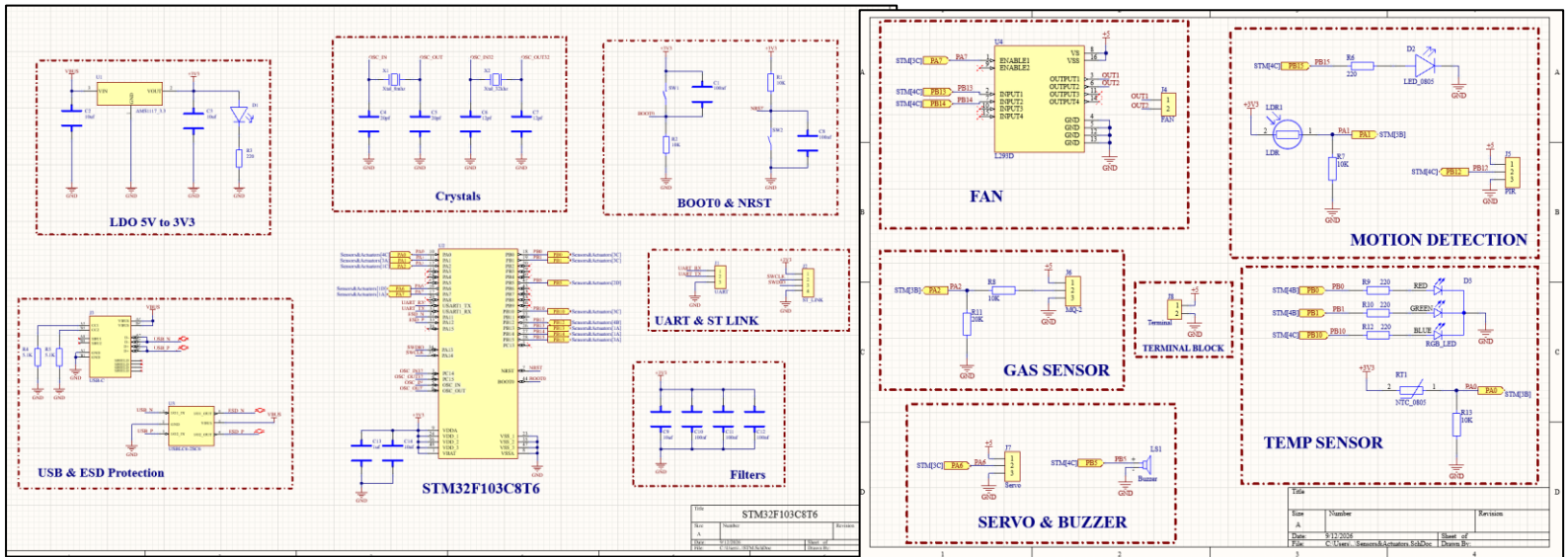
| Design Item ID | Description                                   |
|----------------|-----------------------------------------------|
| AMS1117_3.3    | 1A Low Dropout (LDO) Regulator                |
| Buzzer         | Active Buzzer (Built-in Driving Circuit) 2.3V |
| Cap_0805_1uf   | CAP CER 1uF 50V X7R 0805                      |
| Cap_0805_10nf  | CAP CER 10nF 50V X7R 0805                     |
| Cap_0805_10uf  | CAP CER 10uF 25V X5R 0805                     |
| Cap_0805_12pf  | CAP CER 12pF 50V C0G 0805                     |
| Cap_0805_20pf  | CAP CER 20pF 50V C0G 0805                     |
| Cap_0805_100nf | CAP CER 100nF 50V X7R 0805                    |
| L293D          | PUSH-PULL FOUR CHANNEL DRIVER WITH            |
| LDR            | Photoconductive Cell                          |
| LED_0805       | LED RED 0805 SMD                              |
| NTC_0805       | THERMISTOR NTC 10kΩ 0805                      |
| pin_header_2   | 1kV 1 2.54mm Pin Header 3A 2.5mm Thr          |
| pin_header_3   | 1kV 1 2.54mm Pin Header 3A 2.5mm Thr          |
| pin_header_4   | 1kV 1 2.54mm Pin Header 3A 2.5mm Thr          |
| RES_0805_5.1K  | RES 5.1kΩ ±1% 125mW 0805                      |
| RES_0805_10K   | RES 10kΩ ±0.1% 125mW 0805                     |
| RES_0805_20K   | RES 20kΩ ±0.1% 125mW 0805                     |
| RES_0805_220   | RES 220Ω ±0.1% 125mW 0805                     |
| RGB_LED        | TriColor ChipLED                              |
| STM32F103C8T6  | Medium-density performance line Arm-b         |
| Switch         | SWITCH TACTILE SPST SMD                       |
| Terminal       | TERM BLK 2POS 5.08mm TH                       |
| USB-C          | CONN RCPT Type-C 16POS SMD                    |
| USBL6-2SC6     | ESD DIODE 5.25VWM 17VC SOT-23-6L              |
| Xtal_8mhz      | CRYSTAL 8MHz 20pF SMD                         |
| Xtal_32khz     | CRYSTAL 32.768kHz 12.5pF SMD                  |

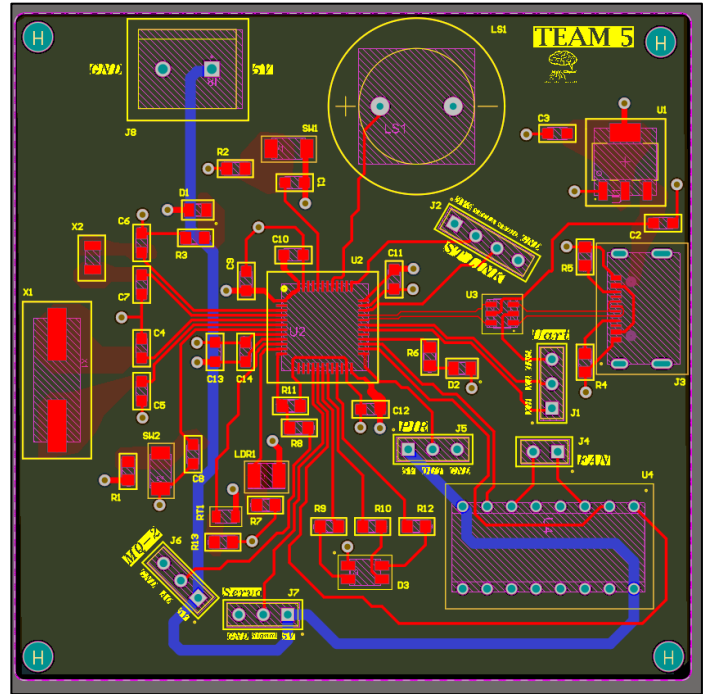
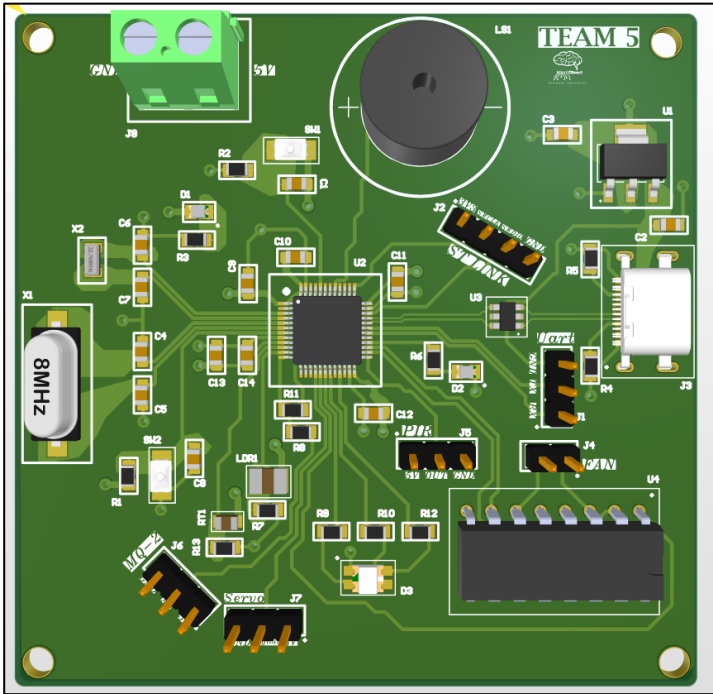
Schematic library

| Name          | Pads | Primitives |
|---------------|------|------------|
| AMS1117       | 4    | 31         |
| BUZZER        | 2    | 14         |
| CAP_0805      | 2    | 16         |
| L293D         | 16   | 32         |
| LDR           | 2    | 24         |
| NTC_0805      | 2    | 12         |
| PIN_HEADER_2  | 2    | 17         |
| PIN_HEADER_3  | 3    | 18         |
| PIN_HEADER_4  | 4    | 19         |
| RES_0805      | 2    | 17         |
| RGB_LED       | 4    | 16         |
| STM32F103C8T6 | 48   | 59         |
| SWITCH        | 2    | 12         |
| TERMINAL      | 2    | 26         |
| USB_C         | 16   | 29         |
| USBL6-2SC6    | 6    | 22         |
| XTAL_8MHZ     | 2    | 12         |
| XTAL_32KHZ    | 2    | 16         |

Pcb library

- Designed a complete 4-layer Printed Circuit Board (PCB) schematic (.SchDoc) and board layout (.PcbDoc) following JLCPCB design rules and constraints.



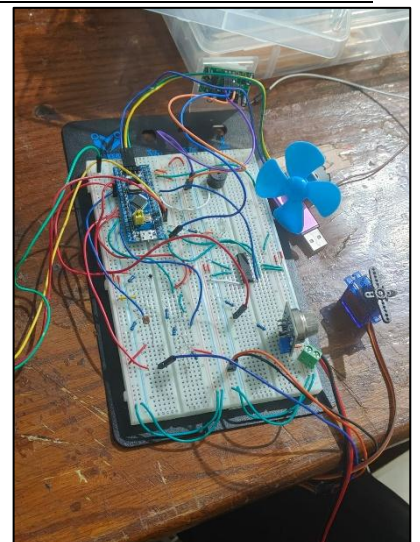


- Successfully passed full Design Rule Check (DRC) with **0 Errors and 0 Warnings**.

| Design Rule Verification Report |                                                              |  |                    |
|---------------------------------|--------------------------------------------------------------|--|--------------------|
| Date:                           | 9/9/2026                                                     |  |                    |
| Time:                           | 10:05:29 PM                                                  |  | Warnings: 0        |
| Elapsed Time:                   | 00:00:00                                                     |  | Rule Violations: 0 |
| Filename:                       | C:\Users\Public\Documents\Altium\mind_project\project.PcbDoc |  |                    |
| Summary                         |                                                              |  |                    |
| Warnings                        |                                                              |  | Count              |
| Total                           |                                                              |  | 0                  |

## Step 5: Final STM32 System Assembly & Embedded Testing

- Assembled the final physical hardware setup around the STM32 microcontroller.
- Validated real-time firmware execution using the Arduino IDE (STM32Duino core).
- Tested sensor response rates, hardware interrupt reliability, and stable multi-actuator control under continuous operational stress.



### **3. Software Development Methodology**

#### **1. Initial Distribution of the Work:**

At the beginning of the Smart Nursery Guardian project, we divided the software work into different parts so that each team member could focus on a specific task. One of the main parts of the project was the Machine Learning (ML) model, while another important part was the software and graphical user interface (GUI).

At first, the team members who were working on the ML part worked separately on developing and testing different models. The goal was to find the model that could give the best results for classifying the baby's cry into the required categories. After comparing the results, the member who achieved the best-performing model continued working on that part and prepared it to be integrated with the rest of the system.

At the same time, two members started working on the software side. One of them focused mainly on preparing and connecting the ML model with the software, while the other focused on building the graphical user interface using Tkinter

#### **2. Machine learning**

##### **Project Development Process**

The project was developed through several stages

---

##### **Testing Audio Feature Extraction**

After recording the first audio file, MFCC features were extracted from it.

The extracted MFCC matrix had the shape:

$13 \times 431$

This means:

- 13 MFCC coefficients were extracted.

- The audio was divided into 431 short frames.

Example values from the extracted features were:

- MFCC 1: approximately -696.2080
- MFCC 2: approximately 90.7658

This stage confirmed that the recorded audio could be converted into numerical features suitable for Machine Learning.

---

### **Extracting Features from the Dataset**

The audio files were processed one by one.

The main operations were:

1. Read the audio file.
2. Process the audio signal.
3. Extract MFCC features.
4. Calculate statistical values from the MFCCs.
5. Convert the audio into a fixed-size numerical feature vector.
6. Attach the correct class label.
7. Store the result in a CSV

file. The first generated feature file was:

audio\_features.csv

The CSV file contained the extracted numerical features and the class label for each audio sample.

This was important because Machine Learning models work with numerical data rather than raw WAV files.

---

### **Training the Initial Random Forest Model**

The first Machine Learning algorithm used was Random Forest.

The first dataset split was:

| Data          | Number of Samples |
|---------------|-------------------|
| Training data | 346               |
| Testing data  | 87                |
| Total         | 433               |

The initial Random Forest accuracy was:

**86.21%**

Although the accuracy was relatively good, the evaluation showed that some classes were not predicted properly.

A warning appeared because some classes had no predicted samples.

This revealed that the model was strongly affected by the imbalance in the dataset.

### **Data Augmentation**

Data augmentation was applied to increase the number of training samples, especially for:

- Tired
- Discomfort

The purpose was to help the model learn more patterns from the minority classes. The augmentation process created additional versions of existing audio samples.

The hungry class did not need the same level of augmentation because it already had a much larger number of samples.

After augmentation, the dataset became:

| Class      | Number of Samples |
|------------|-------------------|
| Hungry     | 305               |
| Discomfort | 84                |
| Tired      | 76                |
| Total      | 465               |

The new dataset was more balanced than the original dataset.

The augmented features were saved in:

training\_features\_augmented.csv

---

### **Testing Random Forest After Augmentation**

Random Forest was tested again using the augmented data. One experiment produced:

100% accuracy

The confusion matrix was:

[[5, 0, 0],

[0, 77, 0],

[0, 0, 5]]

This means that every sample in that specific test split was classified correctly.

The correctly classified samples were:

| Class      | Correct Predictions |
|------------|---------------------|
| Discomfort | 5                   |
| Hungry     | 77                  |
| Tired      | 5                   |

This result showed that data augmentation significantly improved the performance in that particular experiment.

However, the result belonged to one specific split of the dataset.

---

### **Comparing Random Forest and SVM**

Two Machine Learning algorithms were compared:

1. Random Forest
2. Support Vector Machine, or SVM

The comparison used:

- 465 total samples
- 372 training samples
- 93 testing samples

The results were:

| Model         | Accuracy      |
|---------------|---------------|
| Random Forest | <b>88.17%</b> |
| SVM           | <b>62.37%</b> |

Random Forest performed much better than SVM on the available dataset. Therefore, Random Forest was selected as the better model for the project.

---

### **Evaluating Random Forest by Class**

The Random Forest classification report was approximately:

#### **Class      Precision   Recall   F1-score**

|            |      |      |      |
|------------|------|------|------|
| Discomfort | 0.75 | 0.71 | 0.73 |
| Hungry     | 0.95 | 0.97 | 0.96 |
| Tired      | 0.73 | 0.73 | 0.73 |

---

### **Calculating Macro-F1**

Macro-F1 was used because the dataset contained different class sizes.

Macro-F1 gives equal importance to all classes.

The approximate class F1-scores were:

- Discomfort: 0.73
- Hungry: 0.96
- Tired: 0.73

The approximate macro-F1 was:

0.81

This result gives a more balanced view of the model performance than accuracy alone.

---

### **Cross-Validation**

Five-fold cross-validation was performed using Random Forest.

The results were:

#### **Fold Accuracy**

Fold 1 78.49%

Fold 2 81.72%

Fold 3 88.17%

Fold 4 82.80%

Fold 5 82.80%

The average accuracy was:

**82.80%**

The standard deviation was:

**3.12%**

The cross-validation result was important because it tested the model on different parts of the dataset.

It showed that the model performance changed depending on the samples used for training and testing.

The cross-validation result was more representative than relying only on the 100% result from one test split.

---

### **Saving the Trained Model**

After training, the model was saved so that it could be used later without retraining every time.

The model was saved using Joblib.

The saved model used in the prediction system was:

`cry_model_weighted_v3.pkl`

Another trained model file used during the experiments was:

`cry_model_augmented.pkl`

The saved model was then loaded inside the prediction file.

---

### **Creating the Prediction System**

The prediction system was implemented in:

`predict.py`

The main prediction function was:

`detect_cry(audio_file)`

The function receives an audio file and returns:

- The predicted class.
- Confidence values for the classes.
- The detected sound percentage.

The prediction result could be:

- Hungry
- Tired
- Discomfort
- NO\_SOUND

The NO\_SOUND result is produced when the audio does not contain enough detected sound.

---

### **Applying Voice Activity Detection**

Voice Activity Detection was added to reduce incorrect predictions during silence or background noise.

The project used:

WebRTC VAD

The configuration included:

- Sample rate: 16,000 Hz
- Frame duration: 30 milliseconds
- VAD aggressiveness: 2

The audio was divided into short frames.

Each frame was checked to determine whether it contained meaningful sound.

The system then calculated the percentage of frames that contained detected sound.

Examples from testing included:

86.75% sound detected and:

73.49% sound detected

If the sound percentage was too low, the system returned:

NO\_SOUND

This helped prevent the model from classifying silence as a baby condition.

---

## **Noise Reduction**

Noise reduction was also tested as part of the audio processing pipeline. The project used the noisereduce library.

The process included:

1. Recording background noise.
2. Using the background recording as a noise profile.
3. Reducing background noise from the audio.

4. Passing the processed audio to the prediction system.

The recording process included:

- Approximately 3 seconds for the background noise profile.
- Approximately 5 seconds for the main audio recording.

Noise reduction was used to make the audio cleaner before feature extraction and prediction.

---

### **Testing the Model with Dataset Audio**

The model was tested using audio files directly from the dataset. One example was an audio file from the tired class.

The model predicted:

Tired

The confidence values were approximately:

#### **Class      Confidence**

Discomfort 0.67%

Hungry 7.00%

Tired 92.33%

This was a strong correct prediction because the original audio file belonged to the tired class.

---

### **Integrating the ML Model with the Main System**

The Machine Learning model was connected to the main Python application.

The main application was responsible for:

- Starting the system.
- Receiving audio.
- Calling the prediction function.

- Displaying the prediction.
- Displaying confidence values.
- Updating the cry status.
- Triggering the correct response.

The prediction result was passed to the system response logic.

---

## **Final Audio Demonstration**

For the final demonstration, three audio files were used:

hungry.wav

tired.wav

discomfort.wav

The files were processed sequentially.

The sequence was:

1. Play and analyze hungry.wav.
2. Wait 30 seconds.
3. Play and analyze tired.wav.
4. Wait 30 seconds.
5. Play and analyze discomfort.wav.

The expected class was determined from the file name, but the actual result came from the Machine Learning model.

This allowed the team to compare:

- Expected class.
  - Predicted class.
  - Confidence values.
  - Sound percentage.
-

### **3. Integrating the ML Model with the GUI:**

After the initial GUI and ML work were ready, the next major step was connecting the ML model to the graphical interface.

The purpose of this integration was to make the software capable of recording the baby's cry, sending the audio to the ML model, and displaying the resulting prediction and confidence on the GUI.

This stage was completed before connecting the actual hardware. Working on the ML and GUI integration first allowed us to test the complete software side independently and make sure that the model could communicate correctly with the interface.

The simulated sensor values were still used during this stage, which allowed us to see how the interface would look when all the information was displayed together. This helped us avoid waiting for the hardware before testing the software and allowed the hardware work to continue separately.

### **4. Connecting the Hardware Through Serial Communication:**

A few days before the final submission, the team came together to connect the hardware with the software.

The STM32 chip and the connected sensors needed to communicate with the Python application so that their readings could be displayed directly on the GUI. The computer detected the board through COM9, which was then used for the serial communication.

After establishing the connection, we started receiving the real sensor data from the chip and replacing the simulated values with the actual readings. These values were then displayed on the GUI together with the ML prediction.

This was an important stage because it brought the hardware and software parts together into one system. We tested the serial communication several times to make sure that the received values were being read correctly and that the GUI could update them properly.

### **5. Testing and Improvements**

After connecting the main parts, we entered a testing stage. We tested the system several times to find problems in the communication between the different components.

Some tests focused on the ML prediction and making sure that the prediction was displayed correctly. Other tests focused on the sensor values and serial communication. We also tested the GUI to make sure that the displayed information was updated correctly.

Testing was done gradually rather than only at the end of the project. This helped us identify problems earlier and fix them before the final submission.

## **6. Adding Telegram and Video Features**

After the main system was working, we added the additional features that were required to make the project more complete.

The Telegram notification system. The purpose was to allow the system to send an alert when an important situation was detected through a text **CSV** that allows the reading of the username and password without exposing it on the code itself.. This added another way for the user to receive information from the system instead of depending only on the GUI.

We also worked on the video-related part of the project and connected it with the overall system design. These features were added after the main ML, GUI, and serial communication parts were already functioning, so they could be developed without delaying the main system.

## **7. Final Integration**

The final stage was mainly focused on putting everything together and making sure that all parts of the project worked as one system.

The ML model, Tkinter GUI, serial communication, sensors, Telegram notification, and video feature were tested together. We fixed the remaining issues and made several small improvements to the interface and system behavior.

The project went through multiple stages rather than being completed all at once. The early distribution of the work allowed the team members to work independently, while the use of simulation prevented the different parts from blocking each other. Later, when the hardware and software parts were ready, we brought them together through the serial connection and then integrated the ML model.

Finally, during the last testing period before submission, we focused on finding and fixing the remaining problems and making the necessary final adjustments.

## 4. Hardware-Software Integration & Communication

- **UART Serial Bridge:** Configured robust 9600-baud rate UART serial communication bridging the STM32 embedded controller and the Python desktop interface (running on COM9).
- **Bidirectional Messaging Protocol:**
  - **STM32 to Laptop (Sensors & Events):** Periodically transmits structured telemetry strings (DATA:TEMP ,GAS ,MOTIONS ,LIGHT) and event flags (ALERT:GAS\_DETECTED, EVENT:BABY\_AWAKE).

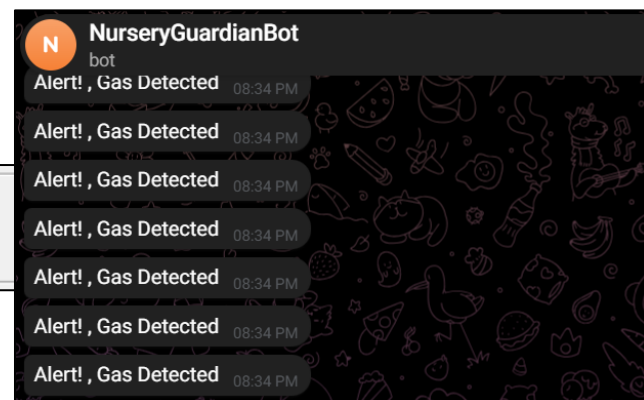
| Sensor Status |          |
|---------------|----------|
| Temperature   | 26.9 °C  |
| Motion        | Sleeping |
| Light         | LIGHT    |
| Gas           | Safe     |

- **Laptop to STM32 (Actuation Commands):** Sends real-time operational commands based on cry classification results (CRY\_DETECTED, CRY\_ENDED, BUZZER\_ON, BUZZER\_OFF).

| Device Status |               |
|---------------|---------------|
| Fan: MEDIUM   | Buzzer: ON    |
| LED: OFF      | Servo: Normal |

- **Telegram IoT Alert Integration:** Integrated secure credential loading (credentials.csv) into Python to launch automated HTTP API calls, triggering instant phone notifications whenever a critical gas leak is detected by the STM32.

| Telegram Alerts |
|-----------------|
| Telegram: Ready |



- **Unified System Execution:** Successfully synchronized sensor polling, real-time GUI updates, Machine Learning predictions, and motor actuation into a single cohesive ecosystem.

## **5. System Flow & Sequence of Operations**

### **1. System Initialization & Startup Sequence**

1. **STM32 Microcontroller Power-On:** Hardware interfaces initialize (ADC channels for Gas/NTC/LDR, GPIOs for PIR, PWM timer for Servo/Fan).
2. **Laptop Application Launch:** Python interface boots up, loads the pre-trained Machine Learning model into memory, and opens the UART serial channel (COM9 at 9600 baud).
3. **GUI Idle State:** Tkinter dashboard activates in a monitoring state while the audio pipeline begins continuous background sampling via the laptop microphone.

### **2. Asynchronous Loop Execution**

- **High-Level Decision Loop (Laptop / Python):**
  1. Captures audio stream continuously in rolling buffers.
  2. Applies spectral noise reduction and Voice Activity Detection (VAD) to filter out ambient noise.
  3. Upon detecting a valid baby cry, sends an immediate CRY\_DETECTED signal to the STM32 to trigger crib rocking via the servo motor.
  4. Extracts MFCC audio features and passes them to the ML classifier.
  5. Updates GUI status and executes tailored response routines:
    - **Hungry:** Displays a calming message and plays response video (hungry.mp4).
    - **Discomfort:** Displays action guidelines (Check diaper, clothing, position).
    - **Tired:** Issues an urgent alert and commands the STM32 to sound the active buzzer (BUZZER\_ON).
  6. Dispatches a CRY\_ENDED command to stop crib rocking once audio returns to baseline.
- **Low-Level Hardware Loop (STM32 Firmware):**
  1. Continuously polls environmental sensors (PIR motion, LDR light, MQ Gas, and NTC Temperature).
  2. Tracks motion frequency; if 4 motion triggers occur within an 8-second window, reports BABY AWAKE to the laptop.
  3. Executes autonomous local temperature control (adjusting fan speed and status LED color based on thermal thresholds).
  4. Evaluates night lighting logic: Illuminates room LED if ambient light is **DARK AND** (BABY AWAKE OR CRY\_DETECTED).
  5. Monitors Gas concentration; if a threshold breach occurs, immediately triggers the local buzzer and sends an alert string to Python to fire a Telegram emergency notification.

## **6. Results & Key Accomplishments**

- **Reliable Sensor Telemetry:** Successfully established stable real-time monitoring of room temperature, light levels, baby movement, and gas concentrations.
- **Precision Gas Safety Alerts:** Verified 100% accuracy in detecting gas hazards, instantly sounding local buzzers while dispatching Telegram alerts to the user's phone.
- **Automated Infant Care Actions:** Validated immediate response routines rocking the crib upon detecting a cry, sounding alarms for urgent tired patterns, presenting clear caregiver guidelines for discomfort, and playing soothing video responses for hungry cries.
- **Production-Ready Hardware Design:** Completed a manufacturable 4-layer PCB design ready for fabrication, transforming a breadboard prototype into an industrial-grade embedded solution.